



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

DIPARTIMENTO DI MATEMATICA LAUREA
TRIENNALE IN INFORMATICA

Anno Accademico 2025/2026

PROGETTO DI BASI DI DATI

BiblioShop Online

Pietro Brocadello, Alvise Greggio

Indice

1	Abstract	1
2	Analisi dei Requisiti	1
3	Progettazione Concettuale	
4	Progettazione Logica	6
4.1	Analisi delle Ridondanze.....	6
4.2	Eliminazioni delle Generalizzazioni.....	8
4.3	Schema Relazionale	10
5	Implementazione in PostgreSQL e Definizione delle Query	11
5.1	Definizione delle Query	11
5.2	Creazione degli indici	13
6	Applicazione Software	14

1 Abstract

Il presente progetto descrive la progettazione e l'implementazione di una base di dati per BiblioShop Online, una piattaforma digitale che combina i servizi di una biblioteca con quelli di un negozio di merchandise. Il sistema opera interamente online e si rivolge a tre categorie di utenti registrati: clienti, venditori e fruitori.

I clienti possono acquistare oggetti disponibili nel catalogo, che comprende libri, DVD, maglie e tazze. Gli acquisti avvengono esclusivamente online e generano una spedizione gestita da un corriere convenzionato e una fattura. I clienti registrati accumulano punti bonus a ogni acquisto e possono utilizzarli per ottenere sconti sugli acquisti futuri.

I fruitori sono utenti iscritti che non effettuano acquisti ma possono prenotare contenuti digitali — libri e DVD in formato elettronico — e usufruirne tramite prestito a tempo determinato. L'iscrizione avviene tramite registrazione al portale, al termine della quale viene assegnata una tessera digitale con relativo numero identificativo. Il prestito di un contenuto è subordinato alla prenotazione dello stesso.

I venditori sono utenti registrati con partita IVA che gestiscono il catalogo degli oggetti in vendita ed emettono le fatture relative agli acquisti. La base di dati tiene traccia dell'intero ciclo di vita di ogni transazione, dalla prenotazione o acquisto fino alla consegna o alla scadenza del prestito.

2 Analisi dei Requisiti

Questa sezione riassume i requisiti a cui deve sottostare la base di dati.

Utenti. Ogni utente per potersi registrare deve fornire le seguenti informazioni:

- **Username** che distingue in modo univoco ogni utente (quindi assumiamo che l'username non possa essere cambiato nel corso del tempo).
- Un **Nome** (dati anagrafici dell'utente)
- Un **Cognome** (dati anagrafici dell'utente)
- Un indirizzo **Email** contatto principale
- Una **Password** per accedere al sito
- **Indirizzo** di residenza, composto dai seguenti campi: via, civico, città, provincia, codice postale, nazione (necessario per gestire spedizioni internazionali).

Un utente al momento della registrazione sceglie una sola categoria (fruitore, cliente o venditore) e non può cambiare categoria né appartenere a più categorie contemporaneamente. Un utente che volesse usufruire di servizi di categorie diverse deve registrarsi con un account separato.

Il **fruitore** è un utente che non acquista ma usufruisce del servizio di prestito digitale.

Al momento della registrazione gli viene assegnata:

- Una **tessera digitale** che viene utilizzata per abilitare il servizio di prestito
- La **data di iscrizione** per sapere da quando viene abilitato il servizio di prestito
- Un **tipo di tessera** (es. standard, studente, anziano). Indica la categoria della tessera, che può determinare i privilegi di prestito

Il fruitore può effettuare prenotazioni di libri e DVD in formato digitale.

Progetto di una Basi di Dati per la gestione di una biblioteca digitale con vendita di merchandise

Il **cliente** è un utente che effettua acquisti online. Oltre agli attributi comuni dell'utente, il cliente possiede:

- Un saldo di **punti bonus**, che viene aggiornato a ogni acquisto in base ai punti guadagnati e ai punti eventualmente utilizzati per ottenere sconti

Il **venditore** è un utente con:

- **partita IVA** obbligatoria per i venditori, necessaria per l'emissione di fatture. Gestisce il catalogo degli oggetti disponibili sulla piattaforma. Il venditore è responsabile dell'emissione delle fatture associate agli acquisti.

Ogni **acquisto** viene identificato dalla data dell'acquisto insieme al cliente che lo ha effettuato, assumendo che un cliente non possa completare due acquisti distinti nella stessa giornata.

I campi che lo compongono sono:

- il **totale** speso per l'acquisto di uno o più oggetti, calcolato tenendo conto degli eventuali sconti tramite punti bonus
- il conto dei **punti guadagnati** dal cliente
- i **punti utilizzati** dal cliente con la quale può avere uno sconto sul totale

Inoltre, ogni acquisto, riguarda uno o più oggetti e, genera automaticamente una fattura.

Trattandosi di acquisti online generano anche una spedizione.

Un **oggetto** è composto da:

- un **codice a barre** che identifica l'oggetto
- il **prezzo** dell'oggetto
- la **giacenza** che indica la quantità disponibile di ciascun oggetto

Il catalogo comprende quattro categorie:

Libro con attributi:

- **autore**: chi lo ha scritto
- **genere**: categoria letteraria

DVD:

- **regista**
- **genere**

Maglia:

- **tessuto** con cui viene realizzata la maglia
- **taglia**: misura del capo
- **colore**: colorazione disponibile

Tazza:

- **colore**: colorazione della tazza
- **dimensione**: misura della tazza
- **materiale** con cui è realizzata la tazza (per es. ceramica, porcellana, vetro)

Prima di ottenere un prestito digitale, il fruitore deve effettuare una prenotazione.

La **prenotazione** ha questi attributi:

- una **data** in cui viene effettuata
- lo **stato**, che indica se la prenotazione è in attesa, confermata o scaduta

Ogni prenotazione viene identificata dal fruitore che l'ha effettuata, dall'oggetto che riguarda la prenotazione e dalla data in cui viene effettuata.

Un **prestito** origina da una prenotazione e registra:

- la **data di inizio** del prestito
- la **data di fine** del prestito

del periodo di accesso al contenuto digitale.

Ogni acquisto produce una **fattura**, ogni fattura viene emessa da un venditore ed è formata da:

- Un **numero fattura**: progressivo identificatore univoco del documento fiscale (obbligatorio per legge)
- **data di emissione**: data in cui la fattura è stata emessa
- **importo totale** della fattura

Progetto di una Basi di Dati per la gestione di una biblioteca digitale con vendita di merchandise

La **spedizione** contiene:

- l'**id** della spedizione: identificatore univoco della spedizione (codice assegnato dal corriere per il tracciamento)
- lo **stato** della **spedizione**: indica la fase attuale della spedizione (es. in transito, consegnata)
- un campo **dettagli**
- un **indirizzo** di consegna composto dai seguenti campi: via, civico, città, provincia, codice postale, nazione.

L'indirizzo di consegna può differire dall'indirizzo di profilo dell'utente, inoltre ogni indirizzo viene identificato dall'insieme di via cap e civico che riteniamo abbastanza univoca. Ogni spedizione è gestita da un **corriere** quest'ultimo è contraddistinto da:

- **nome** denominazione del corriere
- **e-mail**, contatto email del servizio
- **telefono per assistenza**, numero per il supporto clienti
- **package tracking url**: URL per il tracciamento del pacco da parte del cliente

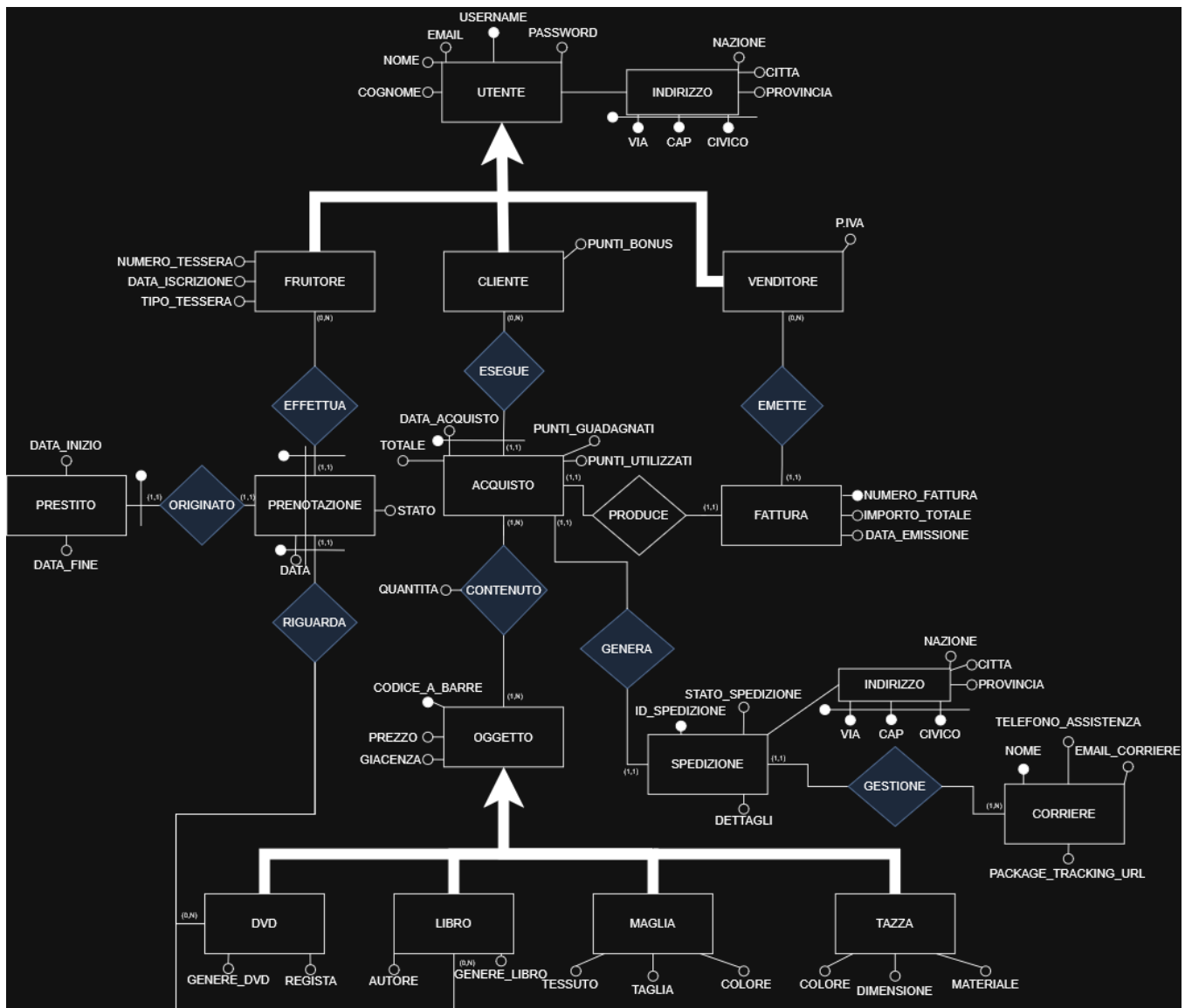


Figura 1: Diagramma E-R della base di dati relativa a BiblioShop Online

3 Progettazione concettuale

Dalla Figura 1, che mostra lo *Schema E-R* risultato dall'analisi dei requisiti, si ricavano le seguenti tabelle che elencano le **entità** e le **relazioni** nel database.

Entità dello schema E-R:

- **UTENTE** — Username, Email, Password, Nome, Cognome, Indirizzo (Via, CAP, Civico, Città, Provincia, Nazione)
- **FRUITORE** — Numero_Tessera, Data_Iscrizione, Tipo_Tessera
- **CLIENTE** — Punti_Bonus
- **VENDITORE** — P.IVA
- **PRENOTAZIONE** — identificata da (Data, relazione **EFFETTUA** con **FRUITORE**, relazione **RIGUARDA** con **DVD/LIBRO**)
- **PRESTITO** — Data_Inizio, Data_Fine
- **ACQUISTO** — Data_Acquisto, Totale, Punti_Guadagnati, Punti_Utilizzati
- **FATTURA** — Numero_Fattura, Importo_Totale, Data_Emissione
- **OGGETTO** — Codice_A_Barre, Prezzo, Giacenza
- **SPEDIZIONE** — ID_Spedizione, Stato_Spedizione, Dettagli
- **CORRIERE** — Nome, Email, Telefono_Assistenza, Package_Tracking_URL
- **DVD** — Genere_DVD, Regista
- **LIBRO** — Autore, Genere_Libro
- **MAGLIA** — Tessuto, Taglia, Colore
- **TAZZA** — Colore, Dimensione, Materiale

Relazioni dello schema E-R:

- **EFFETTUA(FRUITORE, PRENOTAZIONE)**: un fruitore effettua 0:N prenotazioni, e una prenotazione è effettuata da 1:1 fruitore.
- **ESEGUE(CLIENTE, ACQUISTO)**: un cliente esegue 0:N acquisti, e un acquisto è eseguito da 1:1 cliente.
- **EMETTE(VENDITORE, FATTURA)**: un venditore emette 0:N fatture, e una fattura è emessa da 1:1 venditore.
- **ORIGINATO(PRENOTAZIONE, PRESTITO)**: una prenotazione origina 1:1 prestito, e un prestito è originato da 1:1 prenotazione.
- **RIGUARDA(PRENOTAZIONE, DVD/LIBRO)**: una prenotazione riguarda 1:1 contenuto digitale (DVD o LIBRO), e un contenuto digitale può essere riguardato da 0:N prenotazioni.
- **PRODUCE(ACQUISTO, FATTURA)**: un acquisto produce 1:1 fattura, e una fattura è prodotta da 1:1 acquisto.
- **CONTENUTO(ACQUISTO, OGGETTO)**: un acquisto contiene 1:N oggetti, e un oggetto è contenuto in 0:N acquisti. (*attributo: Quantità*)
- **GENERA(ACQUISTO, SPEDIZIONE)**: un acquisto genera 1:1 spedizione, e una spedizione è generata da 1:1 acquisto.
- **GESTIONE(SPEDIZIONE, CORRIERE)**: una spedizione è gestita da 1:1 corriere, e un corriere gestisce 0:N spedizioni.

4 Progettazione Logica

4.1 Analisi delle ridondanze

PUNTI_BONUS in CLIENTE è ridondante perché può essere calcolato sommando tutti i **PUNTI_GUADAGNATI** e sottraendo tutti i **PUNTI_UTILIZZATI** di tutti gli acquisti del cliente tramite la relazione ESEGUE.

TOTALE in ACQUISTO è ridondante perché derivabile moltiplicando il prezzo di ogni oggetto per la quantità acquistata tramite la relazione CONTENUTO, sommando il tutto e sottraendo lo sconto derivante dai PUNTI_UTILIZZATI.

Assunzioni sui volumi:

Concetto	Costrutto	Volume
CLIENTE	E	10.000
ACQUISTO	E	100.000
ESEGUE	R	100.000
OGGETTO	E	500
CONTENUTO	R	300.000

Assunzioni sulle operazioni:

Per PUNTI_BONUS:

- **Operazione 1** (aggiornamento punti dopo acquisto): 500 volte al giorno
- **Operazione 2** (visualizzazione saldo punti, es. ad ogni login o pagina acquisto): 5.000 volte al giorno

Per TOTALE:

- **Operazione 1** (creazione nuovo acquisto): 500 volte al giorno
- **Operazione 2** (visualizzazione totale acquisto, es. storico ordini): 3.000 volte al giorno

Ridondanza 1: PUNTI_BONUS in CLIENTE

Media acquisti per cliente: $100.000 / 10.000 = 10$ acquisti per cliente

CON RIDONDANZA:

Operazione 1 (500/giorno):

Concetto	Costrutto	Accessi	Tipo
ACQUISTO	E	1	S
ESEGUE	R	1	S
CLIENTE	E	1	L
CLIENTE	E	1	S

Operazione 2 (5.000/giorno):

Concetto	Costrutto	Accessi	Tipo
CLIENTE	E	1	L

Costo totale con ridondanza: $500 \times (2 \times 2 + 1 \times 2 + 1) + 5.000 \times 1 = 500 \times 7 + 5.000 = 3.500 + 5.000 = 8.500$

SENZA RIDONDANZA:

Operazione 1 (500/giorno):

Concetto	Costrutto	Accessi	Tipo
ACQUISTO	E	1	S
ESEGUE	R	1	S

Operazione 2 (5.000/giorno), con 10 acquisti per cliente:

Concetto	Costrutto	Accessi	Tipo
CLIENTE	E	1	L
ESEGUE	R	10	L
ACQUISTO	E	10	L

Costo totale senza ridondanza: $500 \times (2 \times 2) + 5.000 \times (1 + 10 + 10) = 2.000 + 105.000 =$
107.000

L'analisi suggerisce quindi di tenere l'attributo ridondante, ottimizzando così il numero di accessi.

Ridondanza 2: TOTALE in ACQUISTO

Media oggetti per acquisto: $300.000 / 100.000 = 3$ oggetti per acquisto

CON RIDONDANZA:

Operazione 1 (500/giorno):

Concetto	Costrutto	Accessi	Tipo
ACQUISTO	E	1	S
CONTENUTO	R	3	S
OGGETTO	E	3	L
ACQUISTO	E	1	L
ACQUISTO	E	1	S

Operazione 2 (3.000/giorno):

Concetto	Costrutto	Accessi	Tipo
ACQUISTO	E	1	L

Costo totale con ridondanza: $500 \times (1 \times 2 + 3 \times 2 + 3 + 1 + 1 \times 2) + 3.000 \times 1 = 500 \times 13 + 3.000 = 6.500 + 3.000 =$
9.500

SENZA RIDONDANZA:

Operazione 1 (500/giorno):

Concetto	Costrutto	Accessi	Tipo
ACQUISTO	E	1	S
CONTENUTO	R	3	S
OGGETTO	E	3	L

Operazione 2 (3.000/giorno), con 3 oggetti per acquisto:

Concetto	Costrutto	Accessi	Tipo
ACQUISTO	E	1	L
CONTENUTO	R	3	L
OGGETTO	E	3	L

Costo totale senza ridondanza: $500 \times (1 \times 2 + 3 \times 2 + 3) + 3.000 \times (1 + 3 + 3) = 500 \times 11 + 3.000 \times 7 = 5.500 + 21.000 = \mathbf{26.500}$

L'analisi suggerisce quindi, anche in questo caso, di tenere l'attributo ridondante, ottimizzando così il numero di accessi.

4.2 Eliminazione delle Generalizzazioni

Nello schema della sez. 2 sono presenti due generalizzazioni:

1. **UTENTE** → **FRUITORE, CLIENTE, VENDITORE** È una generalizzazione **totale** (ogni utente è almeno uno dei tre tipi)
Si è deciso di mantenere **UTENTE** e aggiungere relazioni **IS-FRUITORE, IS-CLIENTE, IS-VENDITORE** poiché
 - Evita la duplicazione degli attributi comuni (Username, Email, ecc.)
 - Riduce i valori nulli rispetto ad accorpare tutto in **UTENTE**
2. **OGGETTO** → **DVD, LIBRO, MAGLIA, TAZZA** È una generalizzazione **totale** (ogni oggetto è esattamente uno dei quattro tipi).
Si è deciso di fare come con **UTENTE** quindi mantenere **OGGETTO** e aggiungere relazioni **IS-DVD, IS-LIBRO, IS-MAGLIA, IS-TAZZA**. Rispetto all'alternativa di accorpare tutti gli attributi specifici direttamente in **OGGETTO** con valori nulli, questa soluzione facilita l'aggiunta di nuovi tipi di oggetto: è sufficiente aggiungere una nuova entità figlia con i propri attributi specifici senza modificare la struttura di **OGGETTO**. Inoltre, il venditore gestisce un catalogo unico di oggetti indipendentemente dal tipo. Avere **OGGETTO** come entità padre permette di interrogare l'intero catalogo con una sola operazione, senza dover unire quattro entità diverse.

4.3 Partizionamento e accorpamento di entità e relazioni

Per quanto riguarda indirizzo si stava valutando se separarlo e collegarlo ad UTENTE e SPEDIZIONE ma alla fine si è deciso di mantenere **INDIRIZZO** come **attributo composto** direttamente dentro UTENTE e dentro SPEDIZIONE perché:

- L'indirizzo di UTENTE e quello di SPEDIZIONE sono indipendenti tra loro in quanto il primo rappresenta la residenza dell'utente e il secondo l'indirizzo di consegna scelto per uno specifico ordine, che può differire dalla residenza
- Mantiene lo schema E-R semplice evitando l'introduzione di relazioni aggiuntive, riducendo la complessità delle query nella fase di implementazione senza alcuna perdita di informazione.

4.4 Schema relazionale

Lo schema ristrutturato presente nella sezione 4.2.1 rappresenta lo **schema logico** in forma visiva. Di seguito vengono elencate tutte le tabelle rappresentabili da tale schema logico:

Utente(Username, Email, Password, Nome, Cognome, Via, CAP, Civico, Città, Provincia, Nazione)

Fruitore(Utente, Numero_Tessera, Data_Iscrizione, Tipo_Tessera)

- Fruitore.Utente → Utente.Username

Cliente(Utente, Punti_Bonus)

- Cliente.Utente → Utente.Username

Venditore(Utente, P_IVA)

- Venditore.Utente → Utente.Username

Prenotazione(Fruitore, Data, Oggetto, Stato)

- Prenotazione.Fruitore → Fruitore.Utente
- Prenotazione.Oggetto → Oggetto.Codice_A_Barre

Prestito(Prenotazione_Fruitore, Prenotazione_Data, Prenotazione_Oggetto, Data_Inizio, Data_Fine)

- Prestito.(Prenotazione_Fruitore, Prenotazione_Data, Prenotazione_Oggetto) → Prenotazione.(Fruitore, Data, Oggetto)

Acquisto(Cliente, Data_Acquisto, Totale, Punti_Guadagnati, Punti_Utilizzati)

- Acquisto.Cliente → Cliente.Utente

Fattura(Numero_Fattura, Importo_Totale, Data_Emissione, Venditore, Acquisto_Cliente, Acquisto_Data)

- Fattura.Venditore → Venditore.Utente
- Fattura.(Acquisto_Cliente, Acquisto_Data) → Acquisto.(Cliente, Data_Acquisto)

Contenuto(Acquisto_Cliente, Acquisto_Data, Oggetto, Quantità)

- Contenuto.(Acquisto_Cliente, Acquisto_Data) → Acquisto.(Cliente, Data_Acquisto)
- Contenuto.Oggetto → Oggetto.Codice_A_Barre

Spedizione(ID_Spedizione, Stato_Spedizione, Dettagli, Via, CAP, Civico, Città, Provincia, Nazione, Acquisto_Cliente, Acquisto_Data, Corriere)

- Spedizione.(Acquisto_Cliente, Acquisto_Data) → Acquisto.(Cliente, Data_Acquisto)
- Spedizione.Corriere → Corriere.Nome

Corriere(Nome, Email_Corriere, Telefono_Assistenza, Package_Tracking_URL)

Oggetto(Codice_A_Barre, Prezzo, Giacenza)

- DVD**(Oggetto, Genere_DVD, Regista)
- DVD.Oggetto → Oggetto.Codice_A_Barre
- Libro**(Oggetto, Autore, Genere_Libro)
- Libro.Oggetto → Oggetto.Codice_A_Barre
- Maglia**(Oggetto, Tessuto, Taglia, Colore)
- Maglia.Oggetto → Oggetto.Codice_A_Barre
- Tazza**(Oggetto, Colore, Dimensione, Materiale)
- Tazza.Oggetto → Oggetto.Codice_A_Barre

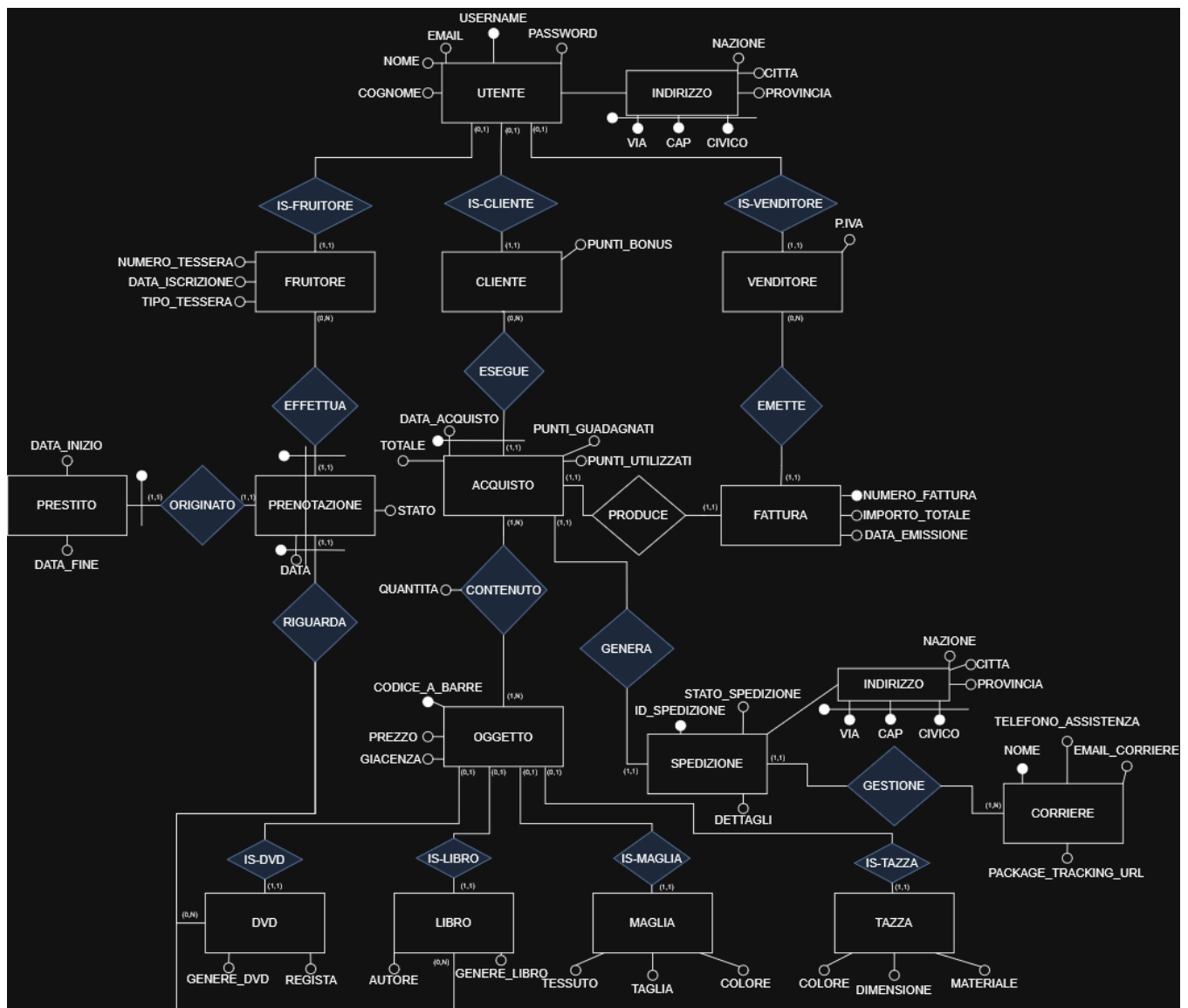


Figura 2: Diagramma E-R ristrutturato.

5 Implementazione PostgreSQL e Query

Il file `biblioshop.sql` contiene il codice SQL usato per la **generazione** delle tabelle, il loro **popolamento** e il codice per la **definizione** delle query utilizzate. Infine, è presente la definizione di un **indice** creato in modo da migliorare le prestazioni di una delle query.

5.1 Definizioni delle query

Query 1: Trovare la top 10 dei clienti che hanno speso di più in totale, mostrando nome, cognome e somma totale spesa, ordinati in modo decrescente.

```
SELECT U.Nome, U.Cognome, SUM(A.Totale) AS TotaleSpeso
FROM Utente U
JOIN Cliente C ON U.Username = C.Utente
JOIN Acquisto A ON C.Utente = A.Cliente
GROUP BY U.Username, U.Nome, U.Cognome
ORDER BY TotaleSpeso DESC
LIMIT 10;
```

	nome character varying (100)	cognome character varying (100)	totalespeso numeric
1	Ornella	De Santis	106.50
2	Elena	Russo	78.78
3	Martina	Ciccone	74.80
4	Irene	Giordano	74.60
5	Lucia	Donati	70.30
6	Chiara	Lobello	64.70
7	Marco	Tagliaferri	57.78
8	Luca	Bianchi	54.78
9	Mario	Rossi	49.40
10	Beatrice	Greco	47.80

Query 2: Trovare i fruitori che hanno effettuato più di una prenotazione, mostrando nome, cognome e numero di prenotazioni, ordinati in modo decrescente.

```
SELECT U.Nome, U.Cognome, COUNT(*) AS
NumeroPrenotazioni
FROM Utente U
JOIN fruitore F ON U.Username = F.Utente
JOIN Prenotazione P ON F.Utente = P.Fruitore
GROUP BY U.Username, U.Nome, U.Cognome
HAVING COUNT(*) > 1
ORDER BY NumeroPrenotazioni DESC;
```

	nome character varying (100)	cognome character varying (100)	numeroprenotazioni bigint
1	Lucia	Donati	2
2	Martina	Ciccone	2
3	Elena	Russo	2
4	Paolo	Conti	2
5	Matteo	Quattrini	2
6	Giulia	Verdi	2
7	Antonio	Ferrari	2
8	Sara	Fontana	2
9	Valentina	De Luca	2
10	Fabio	Mancini	2
11	Mario	Rossi	2
12	Valentino	Sergi	2
13	Luca	Bianchi	2
14	Giuseppe	Santoro	2
15	Chiara	Lobello	2
16	Davide	Marino	2
17	Francesca	Palumbo	2
18	Irene	Giordano	2
19	Carlo	Barone	2
20	Andrea	Brunetti	2

Progetto di una Basi di Dati per la gestione di una biblioteca digitale con vendita di merchandise

Query 3: Trovare i corrieri dove per ogni corriere si trovi il numero di spedizioni gestite e l'importo totale delle fatture associate, ordinati per numero di spedizioni decrescente.

```
SELECT C.Nome AS Corriere,
       COUNT(S.ID_Spedizione) AS NumeroSpedizioni,
       SUM(F.Importo_Totale) AS Totalefatturato
FROM Corriere C
JOIN Spedizione S ON C.Nome = S.Corriere
JOIN Fattura F ON S.Acquisto_Cliente = F.Acquisto_Cliente
              AND S.Acquisto_Data = F.Acquisto_Data
GROUP BY C.Nome
ORDER BY NumeroSpedizioni DESC;
```

	corriere character varying (100)	numerospedizioni bigint	totalefatturato numeric
1	BRT	16	407.52
2	GLS	13	322.27
3	DHL	10	318.34
4	Poste	6	123.57
5	Nexive	5	111.46

Query 4: Trovare gli oggetti più prenotati come contenuto digitale (solo Libri e DVD), mostrando il codice a barre, il tipo (Libro o DVD), l'autore/regista, il genere e il numero di prenotazioni ricevute, ordinandoli per numero prenotazioni.

```
SELECT O.Codice_A_Barre,
       'Libro' AS Tipo,
       L.Autore AS Autore_Regista,
       L.Genere_Libro AS Genere,
       COUNT(P.Oggetto) AS NumeroPrenotazioni
FROM Oggetto O
JOIN Libro L ON O.Codice_A_Barre = L.Oggetto
JOIN Prenotazione P ON O.Codice_A_Barre = P.Oggetto
GROUP BY O.Codice_A_Barre, L.Autore, L.Genere_Libro
UNION ALL
SELECT O.Codice_A_Barre,
       'DVD' AS Tipo,
       D.Regista AS Autore_Regista,
       D.Genere_DVD AS Genere,
       COUNT(P.Oggetto) AS NumeroPrenotazioni
FROM Oggetto O
JOIN DVD D ON O.Codice_A_Barre = D.Oggetto
JOIN Prenotazione P ON O.Codice_A_Barre = P.Oggetto
GROUP BY
O.Codice_A_Barre,
D.Regista,
D.Genere_DVD
ORDER BY
NumeroPrenotazioni
DESC;
```

	codice_a_barre character varying (20)	tipo text	autore_regista character varying (100)	genere character varying (50)	numeroprenotazioni bigint
1	DVD-0009	DVD	Mario Monicelli	Commedia	2
2	DVD-0002	DVD	Federico Fellini	Drammatico	2
3	LIB-0009	Libro	Dino Buzzati	Fantasy	1
4	LIB-0007	Libro	Giovanni Verga	Verismo	1
5	LIB-0005	Libro	Primo Levi	Autobiografia	1
6	LIB-0016	Libro	Leonardo Sciascia	Giallo	1
7	LIB-0003	Libro	Elena Ferrante	Narrativa	1
8	LIB-0021	Libro	Salvatore Quasimodo	Poesia	1
9	LIB-0018	Libro	Carlo Levi	Autobiografia	1
10	LIB-0024	Libro	Ippolito Nievo	Romanzo storico	1

Query 5: Trovare i venditori dove per ogni venditore, bisogna mostrare nome, cognome, numero di fatture emesse e media dell'importo delle fatture, ordinati per media decrescente.

```
SELECT U.Nome, U.Cognome,  
       COUNT(F.Numero_Fattura) AS NumeroFatture,  
       ROUND(AVG(F.Importo_Totale),2) AS MediaImporto  
FROM Utente U  
JOIN Venditore V ON U.Username = V.Utente  
JOIN Fattura F ON V.Utente = F.Venditore  
GROUP BY U.Username, U.Nome, U.Cognome  
ORDER BY MediaImporto DESC  
LIMIT 10
```

	nome character varying (100)	cognome character varying (100)	numerofatture bigint	mediaimporto numeric
1	Logistica	Shop	28	27.66
2	Tutto	Digitale	22	23.12

5.2 Definizioni degli indici

Si sceglie di ottimizzare la QUERY 2, che ricerca e aggrega le prenotazioni per fruitore applicando poi un filtro HAVING. La query esegue le seguenti operazioni: 2 join 1 group by 1 having. Senza indice, ogni esecuzione richiede una scansione completa della tabella Prenotazione per trovare le righe relative a ciascun fruitore (circa N_prenotazioni accessi). Si creano due indici HASH sulle colonne di join, per gli equi-join (JOIN ... ON F.Utente = P.Fruitore):

```
CREATE INDEX idx_prenotazione_fruitore  
ON Prenotazione USING HASH (Fruitore);
```

```
CREATE INDEX idx_fruitore_utente  
ON Fruitore USING HASH (Utente);
```

Inoltre, poiché il GROUP BY raggruppa le tuple per lo stesso valore di Fruitore e il HAVING filtra sul conteggio, un indice B+ Tree sulla colonna Fruitore permette di recuperare direttamente le tuple già ordinate per fruitore, rendendo il raggruppamento più efficiente senza ordinamento aggiuntivo:

```
CREATE INDEX idx_prenotazione_fruitore_btree  
ON Prenotazione (Fruitore);
```

6 CODICE C

Il file *biblioshop.c* contiene il codice necessario ad accedere alla base dati, il quale può essere compilato tramite il comando:

Primo comando per compilare il file .exe (serve avere gcc installato):

```
C:\w64devkit\bin\gcc.exe biblioshop.c -o biblioshop.exe -I.\dependencies\include -  
L.\dependencies\lib -lpq
```

Secondo passaggio per aprire l'eseguibile:
eseguire il file avvia.bat

Bisognerà inserire i dati corretti per connettersi al database:

username DB
password DB
nome DB
IP host
porta

Poi verrà mostrato all'utente un menu con le seguenti opzioni:

```
[-1 ] >> Esci  
[ 0 ] >> Query disponibili  
[1-5] >> Seleziona query
```

tramite cui l'utente potrà: chiudere l'applicazione, visualizzare, in formato testuale, le query disponibili o eseguire una delle cinque query.

Le query sono state **parametrizzate** in modo da dare la libertà all'utente di visualizzare i dati che desidera:

Query 1: Top N ordinati per spesa totale dove N è il numero di risultati da visualizzare (default 10).

Query 2: Fruitori con più di N prenotazioni dove N è soglia minima di prenotazioni (default 1)

Query 3: Top N corrieri per numero spedizioni e totale fatturato dove N è il numero di risultati da visualizzare (default 5)

Query 4: Top N contenuti digitali (Libri e DVD) più prenotati dove N è il numero di risultati da visualizzare (default 10).

Query 5: Top N venditori per numero fatture emesse e importo medio dove N è il numero di risultati da visualizzare (default 2).